

# Research Proposal 0101

## Concurrent Web Server Project

### 1. Project Title & Team

|                      |                                                                                                                            |
|----------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Project Title</b> | Performance Evaluation of Size-Based Request Scheduling in a Multi-Threaded Web Server with Producer-Consumer Architecture |
| <b>Team Members</b>  | Tomas Colarado, Ishit Arhatia, Helena Zhuo, Rudrajit Banerjee                                                              |
| <b>Affiliation</b>   | Northeastern University                                                                                                    |
| <b>Course</b>        | CS5600 - Computer Systems                                                                                                  |
| <b>Instructor</b>    | Logan Schmidt, Ildar Akhmetov                                                                                              |
| <b>Assigned TA</b>   | Chen Zhuge                                                                                                                 |
| <b>Date</b>          | 02/06/26                                                                                                                   |

### 2. Abstract

This project implements a concurrent web server using a fixed-size thread pool with a bounded buffer following the producer-consumer pattern. The main thread accepts incoming HTTP connections and enqueues requests into a shared buffer, while a pool of worker threads dequeue and service requests according to a configurable scheduling policy.

Our research extension investigates **size-based scheduling**; specifically comparing First-Come-First-Served (FCFS) against Shortest-File-First (SFF) to evaluate their impact on mean response time, fairness, and system behavior under varying load conditions. Drawing on established research it demonstrates that web file sizes follow heavy-tailed distributions where most requests are small but most bytes come from few large files [2], we hypothesize that SFF scheduling will significantly reduce mean response time without substantially penalizing large file requests.

We validate this hypothesis through controlled experiments measuring throughput, response time distribution, and the Jain fairness index across different thread pool sizes and workload intensities.

### 3. The Research Extension

#### 3.1 Problem Statement

Traditional web servers process requests in arrival order (FCFS), treating all requests equally regardless of their size. However, this “fair” approach leads to suboptimal average response times due to three factors:

1. **Head-of-line blocking:** Small requests (e.g., 1KB files) get stuck behind large ones (e.g., 1MB transfers), experiencing unnecessary delay.
2. **Workload characteristics:** Web traffic follows heavy-tailed distributions where approximately 50% of requests are under 1KB, yet the largest 1% of files comprise roughly 50% of total bytes transferred [4].

3. **Variance explosion:** Under high load, FCFS provides no mechanism to bound worst-case latency, leading to response time explosion.

### 3.2 Research Question

*Can size-based scheduling (Shortest-File-First) reduce mean response time in a thread pool web server without causing starvation of large file requests, and how does this interact with bounded buffer synchronization?*

### 3.3 Hypothesis

**Primary Hypothesis:** Implementing Shortest-File-First (SFF) scheduling in the bounded buffer will reduce mean response time by a factor of 2–4× compared to FCFS under moderate-to-high system loads ( $\geq 0.7$  utilization), based on theoretical predictions from queueing theory and empirical results from Harchol-Balter et al. [2].

**Secondary Hypothesis:** Counter to intuition, large file requests will *not* experience significant starvation or unfair delays under SFF scheduling. This is because web file sizes follow heavy-tailed Pareto distributions ( $\alpha \approx 1.1$ ), meaning large requests only compete against other large requests—approximately 1% of the total load—rather than the entire workload.

**Tertiary Hypothesis:** The performance gap between FCFS and SFF will widen as system load increases, with SFF maintaining stability while FCFS experiences response time explosion [3].

### 3.4 Specific Algorithm/Mechanism

We will implement two scheduling policies within the bounded buffer:

| Operation        | FCFS (Baseline)          | SFF (Extension)                  |
|------------------|--------------------------|----------------------------------|
| Buffer Structure | Simple queue (FIFO)      | Priority queue by file size      |
| Enqueue          | Insert at tail, $O(1)$   | Insert at tail, $O(1)$           |
| Dequeue          | Remove from head, $O(1)$ | Extract minimum, $O(n)$          |
| Size Lookup      | Not needed               | <code>stat()</code> at admission |

Table 1: Comparison of scheduling policy implementations

The implementation uses the `stat()` system call to obtain file sizes when requests enter the queue. For SFF scheduling, worker threads scan the bounded buffer to find the request with the smallest file size. Given the small buffer size ( $\leq 10$  requests as specified in OSTEP), an  $O(n)$  linear scan is efficient and simpler to implement correctly than a heap-based priority queue.

### 3.5 Theoretical Justification

Our research extension is grounded in three foundational papers:

#### 3.5.1 SEDA Architecture (Welsh et al., 2001)

The Staged Event-Driven Architecture (SEDA) introduced the concept of decomposing servers into stages connected by explicit queues with thread pools [1]. Critically, their Haboob web server implemented **Shortest Connection First (SCF)** scheduling:

“Haboob implements shortest connection first (SCF) scheduling, which reorders the request stream to send short cache entries before long ones.” [1]

Results showed Haboob achieving 201 Mbps throughput (vs. 173 Mbps for Apache), with maximum response time of only 3.9 seconds compared to 93.6 seconds for Apache—demonstrating that size-based scheduling enables both high performance and fairness.

### 3.5.2 Size-Based Scheduling (Harchol-Balter et al., 2003)

This paper provides the theoretical foundation for our hypothesis [2]:

“SRPT-based scheduling decreases mean response time in a LAN by a factor of 3–8 for system load greater than 0.5... 99% of the requests improve by a factor of 10 under SRPT-based scheduling. Only the largest request suffers an increase in mean response time (by a factor of only 1.2).”

The paper explains why large requests are not starved: under heavy-tailed distributions, a request in the 99th percentile of size only competes against 1% of the system load under SFF, whereas it competes against 100% under FCFS.

### 3.5.3 Scheduling Under Overload (Schroeder & Harchol-Balter, 2006)

This paper demonstrates practical implementation in Apache web servers and Linux kernels [3]:

“We find that SRPT scheduling induces no penalties. In particular, throughput is not sacrificed and requests for long files experience only negligibly higher response times under SRPT than they did under the original FAIR scheduling.”

## 3.6 Evaluation Metrics

| Metric                      | Definition                                                                                    |
|-----------------------------|-----------------------------------------------------------------------------------------------|
| Mean Response Time          | Average time from request arrival to completion                                               |
| Max Response Time           | Worst-case latency (fairness indicator)                                                       |
| Throughput                  | Requests completed per second                                                                 |
| Jain Fairness Index         | $f(x) = \frac{(\sum x_i)^2}{N \cdot \sum x_i^2}$ where $x_i$ is requests completed per client |
| Response Time by Percentile | RT as function of file size percentile                                                        |

Table 2: Evaluation metrics for comparing FCFS and SFF

| OSTEP Concept       | Application in Project                 |
|---------------------|----------------------------------------|
| Threads             | Fixed-size worker thread pool          |
| Locks (Mutex)       | Protect shared bounded buffer          |
| Condition Variables | Block on empty/full buffer conditions  |
| Producer-Consumer   | Main thread produces, workers consume  |
| Scheduling Policies | FCFS vs. SFF within the bounded buffer |

Table 3: Mapping of OSTEP concepts to project implementation

## 4. Proposed Scope

The following are the proposed scope for this project.

## 4.1 Multi-threaded

### 1. Implementing Thread Pool

- Update the `wserver.c` script to be able to take in `[-t threads]`.
  - Default is 1 and the executable must fail should it be given a negative number.
- Initialize the thread pool.
  - Loop and create the number of threads that the user desires.
- Create process request function.
  - This function will lock the thread assigned to the work & process the request.
  - Unlock thread whenever it's done with the work to notify the pool is ready for another job.
- Destroy thread pool.
  - The memory allocated must be safely destroyed to have an easy shut down process.

### 2. Implementing Buffer

- Update the `wserver.c` script to be able to take in `[-b buffers]`.
  - Default is 1 and the executable must fail should it be given a negative number.
- Create a global queue data structure buffer.
  - Initialize memory to hold the addresses of the incoming requests in a format that the scheduling policies can process based on time they come in and/or how large they are.

## 4.2 Scheduling Policies

Update the `wserver.c` script to be able to take in `[-s schedalg]`. Pass in either a string for 'FIFO' or 'SFF' or fail out.

### 3. First-in-First-out (FIFO)

- Update the `wserver.c` script to be able to take in `[-s schedalg]`.
  - Pass in either a string for 'FIFO' or 'SFF' or fail out.
- Grab the request at the top of the queue.
- Give the metadata of the request to the next available thread to be processed.

### 4. Smallest File First (SFF)

- Grab the smallest request found in the buffer.
- Give the metadata of the request to the next available thread to be processed.

## 4.3 Security

### 5. Input Parameter Checking (already incorporated in the work above)

- Prevent buffer overrun by checking to see if the user has passed in a valid number for thread size or buffer size.
- All or nothing. If the user doesn't pass in the name of the scheduler correctly, we fail out to avoid any potential malicious inputs.

### 6. Server Timeout

- Be sure to keep track of a timer so that the server doesn't stay on for too long.

## 5. References

### References

- [1] M. Welsh, D. Culler, and E. Brewer, “SEDA: An Architecture for Well-Conditioned, Scalable Internet Services,” in *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP '01)*, Banff, Canada, 2001. <https://www.sosp.org/2001/papers/welsh.pdf>
- [2] M. Harchol-Balter, B. Schroeder, N. Bansal, and M. Agrawal, “Size-based Scheduling to Improve Web Performance,” *ACM Transactions on Computer Systems (TOCS)*, vol. 21, no. 2, pp. 207–233, May 2003. <https://www.cs.toronto.edu/~bianca/papers/submtocs-3.pdf>
- [3] B. Schroeder and M. Harchol-Balter, “Web Servers Under Overload: How Scheduling Can Help,” *ACM Transactions on Internet Technology (TOIT)*, vol. 6, no. 1, pp. 20–52, February 2006. <https://www.cs.cmu.edu/~harchol/Papers/toit06.pdf>
- [4] M. Crovella and A. Bestavros, “Self-Similarity in World Wide Web Traffic: Evidence and Possible Causes,” *IEEE/ACM Transactions on Networking*, vol. 5, no. 6, pp. 835–846, December 1997.